

Lab 3 Report

Group: unnati-rohan

1. KV Server Architecture

Our KV service replicates state through Raft, with three optimizations layered on top: persistent replicator threads, sharded state and leader-lease reads.

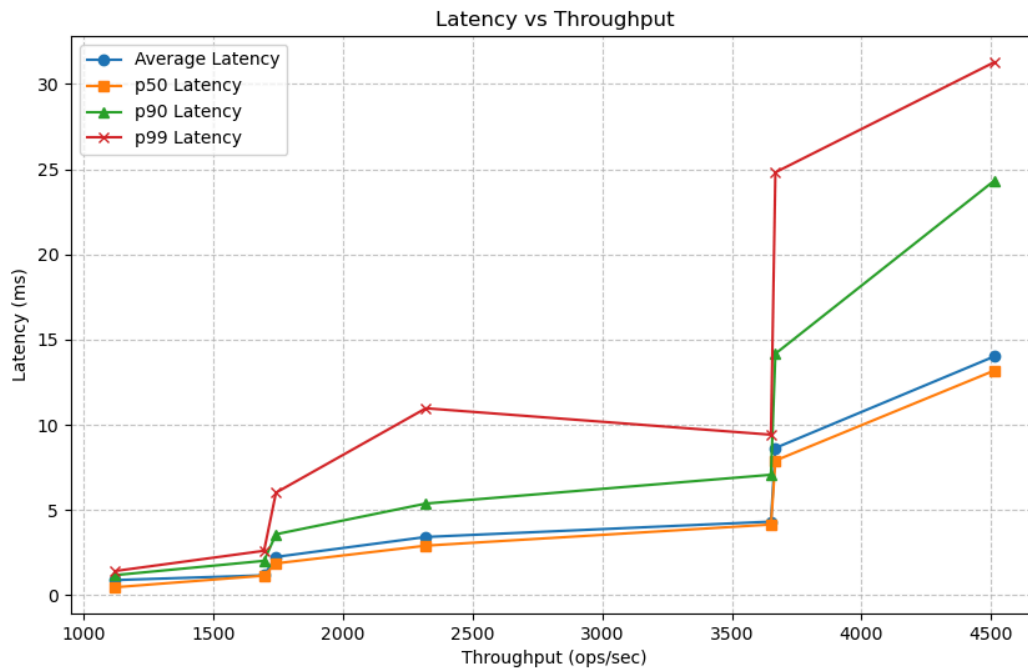
Propose to Apply to Respond. On Put/Append (and non-fast-path Get), `execute_op()` serializes the command (`op | key | client_id | seq_num | value`) and calls `raft_.propose(data)`. It then registers a `PendingOp` in `pending_ops_[index]`. The handler thread blocks on a per-op condition variable for up to 1s. On the Raft side, replicator threads reach quorum, advance `commit_index_`, and signal a dedicated apply thread. Apply drains `commit_index_` to `last_applied_`, calling `kv_server.on_apply()` per entry; `on_apply` applies to the right shard, updates RIFL, and notifies the matching `PendingOp`. The handler wakes and returns the response.

Per-op condition variables. Each `PendingOp` owns its own mutex+cv. With 64 concurrent clients, a single global CV would force every notify to fight for one mutex; per-op CVs let many threads wait independently and only wake the one whose entry was committed.

RIFL duplicate detection. Each shard owns a `rifl_table` mapping `client_id` to `{seq_num, status, value}`. RIFL is enforced inside `on_apply`: when the applied entry's `seq_num` is $\leq$ the cached `seq_num` for that client, `on_apply` returns the cached status/value instead of re-executing. This guarantees idempotence even when a client retries against a new leader after a previous leader's failure, the retry will get the same response the original commit produced.

2. Latency-Throughput Plot (3 replicas)

From tput 64 50 (50% Put / 50% Get) on a 3-replica cluster:

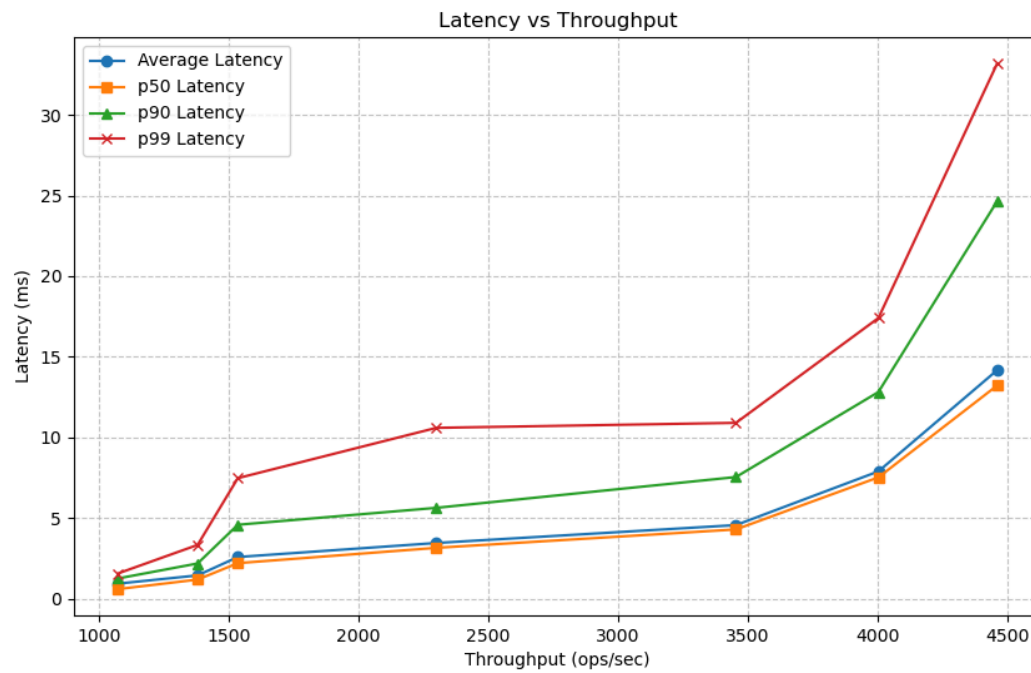


Clients	latAvg (ms)	latP50 (ms)	latP90 (ms)	latP99 (ms)	Throughput (ops/sec)
1	0.89	0.47	1.18	1.42	1,121
8	3.42	2.91	5.38	10.97	2,320
16	4.32	4.15	7.07	9.42	3,651
64	14.01	13.18	24.32	31.27	4,512

On the 90/10 (read-heavy) workload, throughput reaches 7,224 ops/sec at 64 clients (latP50 8.35 ms). The 50/50-vs-90/10 gap is caused by the leader-lease read fast path. The curve shape is classic queue saturation: the latency-throughput product is constant in the 1-8 region, then p50 rises faster than throughput, and p99 grows sharply, the leader's global Raft mutex (log append + commit_index_) is the binding constraint at 64 clients.

3. 5-Replica Comparison

We re-ran the benchmark on a 5-replica cluster (tput5.cpp, identical to tput.cpp except num_nodes = 5).



At 64 clients:

Metric	3 replicas	5 replicas	Delta
Throughput, 50/50	4,512 ops/sec	3,396 ops/sec	-25%
Throughput, 90/10	7,224 ops/sec	5,607 ops/sec	-22%
latP50, 50/50	13.18 ms	16.46 ms	+25%
latP99, 50/50	31.27 ms	45.22 ms	+45%
latP50, 1 client (50/50)	0.47 ms	1.31 ms	+179%

Why 5 replicas is slower. Three factors compound:

- **Larger commit quorum.** Commits need 3-of-5 acks instead of 2-of-3, so `commit_index_` advances on the slower of the two slowest peers. Even with localhost networking, scheduling jitters dominate the tail.
- **More replication traffic.** 4 replicator threads per leader vs 2; doubles per-propose lock work and gRPC fan-out, and each response handler re-acquires the global Raft mutex.
- **More background mutex pressure.** Every replicator periodically reacquires the global mutex to read state. Doubling replicators doubles this cost, significant when the foreground proposal competes for the same lock.

90/10 degrades less than 50/50 (-22% vs -25%) because lease-served reads bypass Raft entirely; only the 10% writes pay the consensus penalty. A subtle but visible effect in Figure 2: 5-replica floor latency at 1 client jumps from 0.47 ms to 1.31 ms, pure consensus overhead, not queueing.

4. Optimizations Attempted

All optimizations below are present in the submitted code. Performance was developed iteratively against the leaderboard.

- **Persistent replicator threads with natural batching.** The original lab patch spawned a fresh `std::thread` per `AppendEntries` RPC and per response handler, ~256 thread spawns per wave at 64 clients. The submitted code runs one long-lived thread per peer (`replicator_loop` in `raft.cpp`) sleeping on a per-peer `condition_variable`; `propose()` just signals it via a `has_work` atomic flag. The replicator drains all log entries between wakeups into a single `AppendEntries` (capped at 1000 entries per RPC), so concurrent proposes naturally batch into one RPC. **Single-client median latency: 20.23 ms to 1.27 ms (~16x faster).**
- **Dedicated apply thread.** Apply runs on its own thread (`apply_loop` in `raft.cpp`) that waits on `apply_cv_`, signaled whenever a replicator response advances `commit_index_`. This decouples apply throughput from RPC handling and avoids holding the Raft mutex during apply work.
- **Leader-lease reads (read-index Get fast path).** Reads through Raft are wasteful when the leader has a fresh quorum ack. After every successful `AppendEntries`, the leader extends `lease_deadline_ns` by 40 ms (shorter than the 50 ms heartbeat period). `KvServer::Get` first calls `raft_.get_read_index()`; if the lease is valid it reads the relevant shard directly and returns, bypassing the `propose+apply` path entirely. **90/10 throughput at 64 clients: ~5,400 to 7,224 ops/sec (+34%).**
- **Sharded KV state (64 shards).** A single global mutex over the store and RIFL table forced unrelated Put/Append to serialize. The submitted code uses `std::array<Shard, 64>`, where each shard owns its own mutex, store map, and RIFL table, chosen via `hash(key) % 64`. `pending_ops_` keeps its own dedicated mutex separately. **50/50 throughput at 64 clients: ~2,500 to 4,512 ops/sec (+80%).**
- **Race-recovery cache (completed_ops_).** `on_apply` could fire before `execute_op` had finished registering `pending_ops_[idx]`, because under low load replication completes faster than the client thread can return from `propose()`. The original behavior dropped the

result and let the client time out at 1 s. The submitted code stashes such results in `completed_ops_[idx]` (capped at 4096 entries); `execute_op` checks this map immediately after registering its pending op and adopts a stashed result if present. No throughput change, but rare 1-second tail outliers disappeared from the latency benchmark.

- **Zero-copy parsing in `on_apply`.** `on_apply` uses `std::string_view` to walk the serialized command and `std::from_chars` to parse `client_id` and `seq_num`, avoiding the per-entry `std::string` allocations and `std::stoull` calls present in the original. Negligible impact on small benchmarks but reduces apply-thread overhead under sustained load.

Final 3-replica numbers vs the lab-patch baseline:

Metric	Baseline	Final	Improvement
Median Put latency (1 client)	20.2 ms	0.47 ms	43x faster
Throughput, 50/50, 64 clients	2,855 ops/sec	4,512 ops/sec	1.6x
Throughput, 90/10, 64 clients	5,397 ops/sec	7,224 ops/sec	1.3x